08
Design an Elevator System

In this chapter, we will explore the object-oriented design of an Elevator System. Compared to some of the other popular interview questions, this one is simple, and it doesn't involve any complex algorithms. Instead, when doing this design, the person will look for a well-organized design that is easy to maintain and extend, the ability to think through edge cases, and the algorithm that determines the elevator's movement.

Requirements Gathering

Here is an example of a typical prompt an interviewer might give:

"Design an elevator system. Assume there are a fixed number of elevator cars, and they all go to the same set of floors. You should be able to call an elevator car to your floor, and then ride it to your destination floor. Once in the elevator, you should be able to select which floor to go to, and the elevator car will move to that floor. Behind the scenes, the system efficiently manages elevator assignments and optimal movement."

Requirements gathering is an important part of the design process. Here are some questions that might be asked:

Here is an example of how a conversation between a candidate and an interviewer might unfold:

Candidate: Do we need to consider power-on for office building, or do we need to consider other types of elevators as well, such as industrial elevators for factories or freight elevators for heavy goods?

Interviewer: Let's focus on passenger elevators.

Candidate: Do all elevator cars have the same set of floors?

Interviewer: Yes, all elevator cars stop at the same floors.

Candidate: When a user presses the button for "Down" from a particular floor, what strategy should the system use to determine which elevator to dispatch?

Interviewer: Assume there are two cars, one in the 5th floor, moving up, and a second one, stopped, waiting for a request. The system should dispatch the second elevator because it is closer to the floor where the request is made.

Candidate: Should the system be optimized for a fleet of elevators dispatch to the elevator's current movement, the request origin, or the destination?

Interviewer: Focus on the request origin.

Candidate: Should the system be configurable, allowing the system to easily switch between different optimization strategies.

Interviewer: Yes, that's a great idea. Assume we want to customize sensor to elevator systems. It's a great idea to let them briefly, during an elevator's idle time, scan the floors it is responsible for. This allows the elevator to know if there is a request or waiting on a specific floor, and it can adjust its movement accordingly. Let's focus on designing the elevator's algorithm.

> ✅ Step This has thrown people off when they have asked, and the interviewer then should only have an "ear" listener. While it's great to recognize this dark design detail, it's not critical if it isn't the primary focus.

Requirements

Based on the conversation and how elevator systems work in the real world, here are the key functional requirements we will implement:

- The system manages multiple elevator cars, all of which serve the same set of floors.
- Users can press "Up" and "Down" buttons that prompt a specific floor and whether an hallway getting.
- Each elevator car should display its current floor and state (e.g., moving up, moving down, or idle).
- Inside each elevator, passengers can select their destination floor, and the system should route the car across the bottom corresponding to the floor they want to go.
- If no car is ready the elevator car arrives at floor number a destination opposite the elevator's current movement, the request should be queued.

Here are some non-functional requirements:

- The dispatching algorithm should be configurable, allowing the system to easily switch between different optimization strategies.

Some of the requirements above are based on common sense in elevator systems. It's a great idea to list these briefly, acting as a checklist for the design. For example, the last bullet point, "If no car is ready, the request should be queued," is not obvious at starting, but it's important to include in the design.

Understanding elevator control panels

Elevator systems typically include two types of buttons, each serving a distinct purpose for controlling elevator operations:

- **Hallway Buttons (Outside the Elevator):** Hallway buttons are located on each floor outside the elevator. These buttons, sometimes an "up" button to call the elevator to go up and a "down" button to call the elevator to go down.
- **Floor Buttons (Inside the Elevator):** Floor buttons are located on the control panel inside the elevator. Each button corresponds to a specific floor.

By looking at the two diagrams, we can understand how passengers interact with the elevator system. Hallway buttons initiate a request to the elevator system, while floor buttons allow passengers to specify their destination floor.

Use Case Diagram

A use case diagram visually illustrates the various scenarios a person interacts with a system to achieve specific goals in the elevator system. We will keep our hallway actions, such as requesting an elevator, selecting a floor, and dispatching an elevator.

Use Case Diagram of Elevator System

The use cases for the **Passenger** actor are as follows:

- **Request Elevator:** This represents the action of a passenger on a specific floor pressing the hallway button to request an elevator.
- **Select Floor:** Once inside the elevator, a passenger selects their destination floor using the elevator's floor buttons.

The use cases for the **System** actor are as follows. Both these actors are not necessarily human:

- **Dispatch Elevator:** This represents the system assigning the most appropriate elevator to a request based on availability and suitability.
- **Request Elevator Status:** This represents the elevator's motion frequently sends an update and other parts needed.

Identify Core Objects

Before diving into the design, it's important to enumerate the core objects.

- **Elevator System:** This is the facility that provides the main interface to the elevator system. It coordinates the overall operation by tracking the status of all elevator cars and delegating hallway calls (requests to the Elevator Dispatcher for optimal assignment.
- **Elevator Dispatcher:** This component is responsible for assigning the most suitable elevator car to a hallway call, considering factors such as current location, direction, and the available elevators using the dispatching strategy and select the most suitable elevator based on the current state and the request's location.
- **Elevator Car:** This represents a single elevator car. It maintains its current floor, direction, and a list of pending stops. Each car operates independently, responding to commands from the Elevator Dispatcher and managing its own movement and door operations.

Design Class Diagram

To choose the right strategy for modeling our system, let's first consider whether the elevator problem is more centered around data or behavior. While elevators do have state (current floor, direction, and pending stops), the core challenge is coordinating multiple cars to efficiently handle requests. Since the key design decisions revolve around how to dispatch requests and optimize movement, we lean toward the Elevator Dispatcher, Individual Elevators, or Passenger.

Since the use cases are more about the coordinating elevator needs, we'll start with the system's behaviors and user interactions (represented through use cases) and then use them to guide the definition of classes and methods. While the core use cases already identify, we can now refine the interfaces and relationships between the Elevator System, Elevator Dispatcher, and Elevator Car.

Elevator System

The Elevator System class serves as the central controller, providing an API for controlling all the elevator cars, hosting their states, and handling dispatching requests efficiently.

By looking at the use cases, we can identify the core responsibilities of the system, which lead to specific APIs:

- **Set status:** Check the current state of each elevator (e.g., which floor it is on, and its direction of movement). This is used in the dispatch logic to determine which elevator is best suited to handle a request.
- **Request elevator:** When a user presses a hallway button on a floor, the system must assign an elevator to that request.
- **Select floor:** The Elevator Dispatcher calls this elevator, passes the floor button on the control panel for their destination floor.

The Elevator System's Dispatcher delegates the actual dispatch logic to the Elevator Dispatcher, ensuring the system can easily switch between different optimization strategies.

Elevator Car

The Elevator Car class models the behaviors of an elevator car within the system. It maintains a queue of target floors to track pending stops and manages its movement between floors while responding to passenger requests.

The delegation allows the Dispatcher class to encapsulate dynamic attributes, such as the elevator's current floor and direction, without cluttering the Elevator Car with dispatch-specific logic. By defining interfaces **IE** (**IEliveCar**, **IEDispatcher**), the UML diagram more illustrates the relationships.

Elevator System

The ElevatorSystem class serves as the central controller, providing an API for controlling all the elevator cars, hosting their states, and handling dispatching requests efficiently.

By looking at the use cases, we can identify the core responsibilities of the system, which lead to specific APIs:

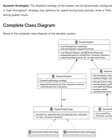
- **Set status:** Check the current state of each elevator (e.g., which floor it is on, and its direction of movement). This is used in the dispatch logic to determine which elevator is best suited to handle a request.
- **Request elevator:** When a user presses a hallway button on a floor, the system must assign an elevator to that request.
- **Select floor:** The Elevator Dispatcher calls this elevator, passes the floor button on the control panel for their destination floor.

The Elevator System's Dispatcher delegates the actual dispatch logic to the Elevator Dispatcher, ensuring the system can easily switch between different optimization strategies.

Elevator Car

The Elevator Car class models the behaviors of an elevator car within the system. It maintains a queue of target floors to track pending stops and manages its movement between floors while responding to passenger requests.

The delegation allows the Dispatcher class to encapsulate dynamic attributes, such as the elevator's current floor and direction, without cluttering the Elevator Car with dispatch-specific logic. By defining interfaces **IE** (**IEliveCar**, **IEDispatcher**), the UML diagram more illustrates the relationships.



Alternative approach: We could have used a generic data structure, such as a map/dictionary, to store elevator states. This approach would enable the system to easily add new elevator properties without modifying the core class. However, it would sacrifice strong type enforcement, e.g., implemented statically the added efficiency offers cognitive overhead.

Direction

The Direction enum provides a type-safe way to represent elevator movement directions. It includes three possible states:

- **UP:** The elevator is moving upward.
- **DOWN:** The elevator is moving downward.
- **IDLE:** The elevator is stationary.

By using an enum instead of raw strings or integers, the system minimizes ambiguity and ensures consistent, predictable behavior across all elevator cars. It plays a key role in optimizing elevator operations, helping prevent unnecessary direction changes, and ensuring clear communication between components.

The system can easily extend this enum with additional directions if an elevator model requires more complex movement, such as elevators with multiple doors or elevators that cover diagonal.

- The system can prioritize elevators already moving toward the requested floor, reducing passenger wait times.
- The UML diagram for the Direction enum is shown below.

Elevator Dispatch and Dispatching Strategy

The ElevatorDispatch class plays a critical role in managing user requests from hallway buttons and floor buttons, determining and selecting the appropriate elevator car to handle each request efficiently. To clarify what "dispatch" means in this context, it refers to assigning an elevator to handle a hallway call request and directing it to make a stop. From the perspective of an elevator car, this means adding the requested floor to its queue of pending stops.

The dispatch logic relies on the DispatchingStrategy interface, which allows the system to easily switch and swap different algorithms for optimizing elevator allocation.

The system supports three primary dispatch strategies, each suited to different use cases:

- **Nearest Idle Elevator Dispatch:** This strategy assigns the closest idle elevator to a hallway call, minimizing initial response time. It works well in low-traffic scenarios but may lead to inefficiencies in busy periods if multiple elevators are available, the system may end up dispatching
- **Direction-aware:** This strategy is already moving in the requested direction, it may be prioritized.
- **Minimizing total travel time:** This strategy calculates the overall expected wait time for passengers.

The system can easily add more strategies, such as zone-based dispatching or predictive dispatch, by implementing new classes that adhere to the DispatchingStrategy interface. This design ensures the dispatch logic remains flexible and adaptable to different building layouts and traffic patterns.

Below is the implementation of the class:

- dispatchElevatorCarUpdate(int, Direction) and dispatchElevatorCarUpdate(int): Processes the request from a hallway button by evaluating the current state of all elevators and assigning the most suitable elevator car.

DispatchingStrategy Interface

The DispatchingStrategy interface specifies methods for selecting an elevator car when a hallway button is pressed.

By abstracting the selection logic, the system is adaptable to various strategies, allowing flexibility in optimizing the dispatch process.

Common dispatching strategies

First Come, First Serve (FCFS): The system assigns the request to the first available elevator in the system's dispatch queue, regardless of its direction or proximity to the request. This strategy is simple but might not always be the most efficient in a busy building.

Nearest Idle Elevator: This strategy prioritizes elevators that are currently idle and assigns the closest one to the requested floor. This approach reduces wait times by minimizing the distance the elevator must travel to reach the passenger.

Direction-aware: This strategy considers both the elevator's current direction and the passenger's intended direction. It favors elevators in the same direction as the passenger's request, ensuring a smoother experience by avoiding unnecessary direction changes.

Dynamic Strategies: The dispatch strategy of the system can be dynamically configurable based on traffic patterns. For example, the system might switch to a strategy optimized for speed during busy periods, while using a "First Come, First Served" strategy at off-peak times.

Complete Class Diagram

Below is the complete class diagram of the elevator system:

ElevatorSystem
+ CreateElevator()
+ ElevatorCheckStatus(targetFloor)
+ ElevatorCarUpdate(elevatorId)
+ ElevatorCarUpdate(elevatorId)
+ ElevatorCarSelectFloor(elevatorId, targetFloor)
+ requestElevator(requestFloor, requestDirection)
+ selectFloorInElevator(car, int destinationFloor)
ElevatorDispatch
- DispatchingStrategy strategy
+ ElevatorCarUpdate(elevatorId, Direction direction)
+ ElevatorCarUpdate(elevatorId)
DispatchingStrategy
+ findElevatorCarToDispatch(int, int, Direction, ElevatorCar[], ...)
ElevatorCar
+ currentFloor
+ Direction currentDirection
ElevatorCarUpdate
ElevatorCarUpdate
ElevatorCarUpdate

Below is the implementation of the observer pattern for handling hallway call requests:

```
// Observable Subject: HallwayButtonPanel
public class HallwayButtonPanel {
    private final int floor;
    private final List<ElevatorObserver> observers;

    public HallwayButtonPanel(int floor) {
        this.floor = floor;
        this.observers = new ArrayList<>();
    }

    // Handles button press event and notifies all registered observers
    public void pressButton(Direction direction) {
        notifyObservers(direction);
    }

    // Registers a new observer to receive button press notifications
    public void addObserver(ElevatorObserver observer) {
        observers.add(observer);
    }

    // Notifies all registered observers about the button press
    private void notifyObservers(Direction direction) {
        for (ElevatorObserver observer : observers) {
            observer.update(floor, direction);
        }
    }
}

// Observer Interface
public interface ElevatorObserver {
    void update(int floor, Direction direction);
}

// Observer Implementation: ElevatorDispatchController
public class ElevatorDispatchController implements ElevatorObserver {
    @Override
    public void update(int floor, Direction direction) {
        // Logic to handle the floor request
    }
}
```

Why it's better:

- **Decoupled architecture:** The Observer Pattern separates the hallway buttons and dispatch logic, making it easier to maintain, test, and extend the system.
- **Faster response time during rush hours:** By treating hallway button presses as discrete events, the system bypasses any queue buildup during rush hours, ensuring that requests are handled without noticeable delays.

Handling elevators serving different floor sets

In the existing design, all elevators can stop at every floor. What if the building has certain elevators that only stop at specific floors?

In order to fulfill such a requirement, the design should allow each elevator car to be assigned a defined set of accessible floors and ensure the system assigns hallway button requests only to elevators that can stop at the requested floor.

How It Works:

- **Defining accessible floors:** Each elevator car will have a list of floors it can serve. This can be set during the system initialization.
- **Validating hallway call requests:** Before an elevator is assigned or a hallway call request is added to its queue, the system checks whether the requested floor is accessible by the elevator.
- **Updating dispatching logic:** The dispatching strategy must ensure that only elevators that can reach the requested floor are considered for assignment.

Example Scenario: Suppose the building has 20 floors. **Elevator 1** serves only floors 1, 5, 10, 15, and 20. **Elevator 2** serves all the floors. When a user on the 3rd floor presses the "up" button, the system avoids assigning **Elevator 1** since it cannot stop at floor 3. **Elevator 2** will be chosen instead.

Design and code changes:

Update the ElevatorCar class:
We introduce the accessibleFloors attribute to store the list of floors each elevator can serve.

```
// Set of floors this elevator can service
private final Set<Integer> accessibleFloors;
```

Modify the addFloorRequest method:
Before adding a floor to the elevator's list of stops, check whether the floor is part of the elevator's accessible set.

```
public void addFloorRequest(int floor) {
    // Only add the request if the floor is accessible by this elevator and not already in the
    // queue
    if (accessibleFloors.contains(floor) && !targetFloors.contains(floor)) {
        targetFloors.offer(floor);
        updateDirection(floor);
    }
}
```

Update dispatching strategies:
The DispatchingStrategy implementations will ensure that only elevators capable of reaching the requested floor are considered for assignment. For example, in the **Shortest Seek Time First** strategy, add an additional check:

```
public class ShortestSeekTimeFirstStrategy implements DispatchingStrategy {
    @Override
    public ElevatorCar selectElevator(List<ElevatorCar> elevators, int floor, Direction direction) {
        ElevatorCar bestElevator = null;
        int shortestDistance = Integer.MAX_VALUE;

        for (ElevatorCar elevator : elevators) {
            // Calculate distance between elevator and requested floor
            int distance = Math.abs(elevator.getCurrentFloor() - floor);
            // Select elevator if it's idle or moving in the same direction and closer than the
            // current best
            if (elevator.isIdle() || elevator.getCurrentDirection() == direction)
                    // Only consider elevators that can actually reach the requested floor
                    && elevator.getAccessibleFloors().contains(floor)
                    && distance < shortestDistance) {
                bestElevator = elevator;
                shortestDistance = distance;
            }
        }

        return bestElevator;
    }
}
```

Wrap Up

In this chapter, we designed an Elevator System by following a structured approach, similar to how a candidate would solve this problem during an OOD interview. We began by gathering and clarifying requirements through a series of questions and answers with the interviewer. This was followed by identifying the core objects involved, designing the class diagram, and implementing key components of the system.

A key takeaway from this design is the importance of modularity and clear separation of concerns. Each component, such as ElevatorSystem, ElevatorCar, ElevatorDispatch, and DispatchingStrategy, focuses on a specific responsibility, ensuring that the system is maintainable, scalable, and flexible. This modular design allows the system to easily adapt to different dispatching strategies and optimize performance for various building traffic conditions.

In the deep dive section, we explored advanced topics, including using the Observer Pattern for event-driven hallway call requests, where pressing the hallway buttons instantly notifies the dispatch controller, enabling faster elevator assignments. We also discussed handling elevators that serve different sets of floors.

Congratulations on getting this far! Now give yourself a pat on the back. Good job!

Further Reading: Observer Design Pattern

This section gives a quick overview of the design patterns used in this chapter. It's helpful if you're new to these patterns or need a refresher to better understand the design choices.

Observer design pattern

The Observer is a behavioral pattern that lets you define a subscription mechanism, allowing multiple objects to receive notifications and updates automatically whenever the object they are observing changes state.

In the elevator system design, we use the Observer pattern to decouple hallway button presses from the dispatch controller, enabling efficient event-driven request handling. To illustrate the Observer pattern in another domain, the following example uses a news application.

Problem

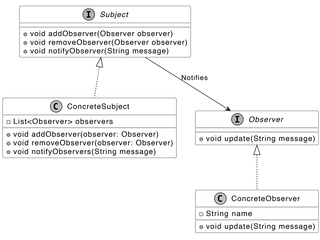
Imagine you're developing a news application that delivers real-time updates to its users. Whenever a breaking news story is published, all users who have subscribed to that category should receive immediate notifications. Implementing this functionality can be challenging as directly linking the news publisher to each user would result in a tightly coupled design, making the system rigid and difficult to maintain. Additionally, as the user base grows, the system must efficiently manage the distribution of updates without becoming a bottleneck.

Solution

The Observer design pattern offers an elegant solution to this problem by establishing a one-to-many relationship between the publisher (news provider) and subscribers (users).

In this pattern:

- The "subject" (or "publisher") is the news application, which holds the core business logic and the breaking news content.
- The "observers" (or "subscribers") are the users who have subscribed to specific news categories and need to be notified when new content is published in those categories.
- The news application (subject) maintains a list of subscribed users (observers), and when a new breaking news story is published (state change), it notifies all subscribed users (observers) by calling an update method on each.



Observer design pattern class diagram

When to use

The Observer design pattern is particularly useful in scenarios where:

- Changes in one object require notifying other objects, especially when the set of objects that need to be notified isn't known in advance or can change dynamically.
- When objects in your application need to observe other objects (subjects), but only under specific conditions or for a limited time.

Mark as Complete

Partner With Us

Be an Affiliate

Become a Contributor

Company & Legal

Our Team

Newsletter

Privacy Policy

Terms of Service

Do Not Sell or Share My Personal Information

Support

hi@bytebytego.com

Report a Bug

Resources

YouTube

Visual Dev Guides

Find Jobs

Prepare for Coding Interviews

Ask Alex

Copyright ©2022-2026 ByteByteGo Inc. All rights reserved.